USAC / FACULTAD DE INGENIERÍA

# ESTRUCTURAS DE DATOS - x

Día, fecha: xx xx xxxx
Hora de inicio: 17:20
Auxiliar: [por completar]
Semana: [por confirmar]

Escuela de Ingeniería de Ciencias y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

## UNIDAD 5

**HEAPS · ORDENAMIENTO Y BÚSQUEDA**

# Unidad 5

## Heaps, ordenamiento y búsqueda

Definición de Heaps · Tipos de Heaps
Algoritmos de ordenamiento · Algoritmos de búsqueda
Aplicación de algoritmos

# Anuncios

xxxx
xxxx
xxxx
xxxx

Completar con los avisos oficiales del curso.

Valor: Tolerancia, 5 minutos.
Área de conocimiento: 55 minutos.

# AGENDA

## Inicio de la sesión

Tolerancia: 5 min.
Definición de Heaps: 10 min.
Tipos de Heaps: 10 min.

## Desarrollo

Algoritmos de ordenamiento: 15 min.
Algoritmos de búsqueda: 10 min.
Aplicación de algoritmos: 10 min.

Conocimiento: 55 min.
Sesión completa: 60 min.

# Valor de la semana: Tolerancia

## Descripción y relación con la sesión

Aceptar las diferencias sin discriminar.

En la sesión se aplica al escuchar distintas soluciones, respetar los ritmos de aprendizaje y comparar algoritmos mediante argumentos, sin descalificar a quienes proponen otra estrategia.

## Aplicación del valor

Actividad de 5 minutos:
1 min: explicar el valor.
3 min: comparar en parejas dos maneras de ordenar una lista y escuchar sus razones.
1 min: compartir un acuerdo.

Aceptar diferencias permite debatir; la corrección del algoritmo se verifica con evidencia.

# Competencias de la planificación

## Primer campo visible

«Aplica técnicas de hashing y árboles de Merkle mediante funciones hash y estructuras criptográficas para garantizar inte…»

## Segundo campo visible

«Desarrolla soluciones basadas en blockchain implementando estructuras encadenadas y funciones hash criptográficas p…»

Fragmentos de la foto: texto recortado y referido a otro contenido. Revisar las competencias en la planificación original.

# 2.1 Contenido

| No. | Unidad | Tema | Sub Tema |
|---|---|---|---|
| 1 | 5 | Heaps | Definición de Heaps |
| 2 | 5 | Heaps | Tipos de Heaps |
| 3 | 5 | Algoritmos de ordenamiento y búsqueda | Algoritmos de ordenamiento |
| 4 | 5 | Algoritmos de ordenamiento y búsqueda | Algoritmos de búsqueda |
| 5 | 5 | Algoritmos de ordenamiento y búsqueda | Aplicación de algoritmos |
| 6 | | | |
| 7 | | | |

# HEAPS Y ALGORITMOS

## Organizar, ordenar y buscar

Desarrollo de los cinco subtemas indicados en la planificación

# 1. Qué es un heap

## Concepto

Un heap binario es un árbol binario completo que cumple una relación de prioridad entre cada nodo y sus hijos. Esa combinación permite mantener un extremo, mínimo o máximo, en la raíz.

Su estructura no exige ordenar todas las claves de izquierda a derecha.

## Ejemplo y análisis

Si el objetivo es obtener repetidamente el elemento de mayor prioridad, basta con consultar la raíz y restaurar la propiedad después de los cambios.

En esta sesión «heap» significa la estructura de datos, no la región de memoria dinámica que también recibe ese nombre.

# 2. La forma de árbol binario completo

## Concepto

Un árbol binario completo llena todos sus niveles excepto, posiblemente, el último. En el último nivel, los nodos se ubican de izquierda a derecha sin dejar huecos intermedios.

Esta forma mantiene la altura en O(log n) y permite representar el árbol en un arreglo.

## Ejemplo y análisis

Con 6 nodos:
Nivel 0: 1 nodo.
Nivel 1: 2 nodos.
Nivel 2: 3 nodos, en las primeras posiciones disponibles.

«Completo» no significa «perfecto»: en un árbol perfecto todos los niveles están totalmente llenos.

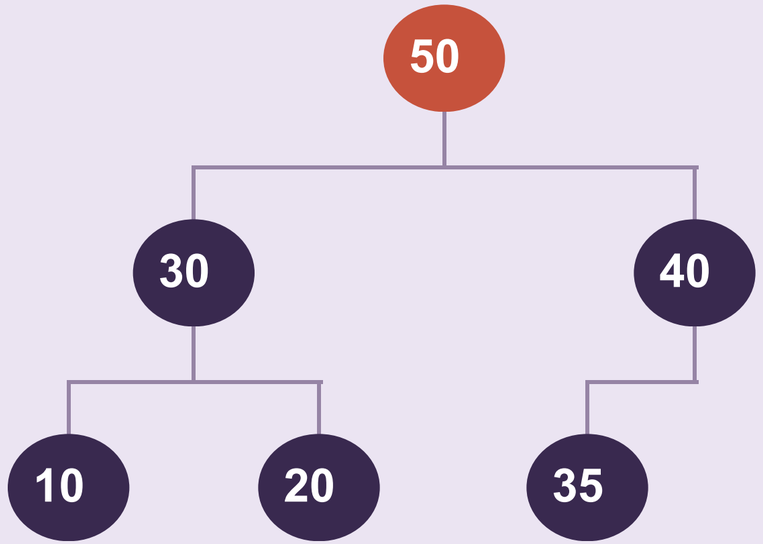
# 3. La propiedad de prioridad

## Concepto

En un max-heap, cada padre es mayor o igual que sus hijos. En un min-heap, cada padre es menor o igual que sus hijos.

La regla se comprueba en todas las relaciones padre-hijo. No se exige una relación entre hermanos ni entre nodos de ramas diferentes.

### Ejemplo y análisis




**Arreglo: [50, 30, 40, 10, 20, 35]**

# 4. Representación del heap en un arreglo

## Concepto

El arreglo almacena el árbol por niveles. La raíz ocupa el índice 0 y los hijos se identifican mediante fórmulas.

Para un nodo en i:
Izquierdo = 2i + 1.
Derecho = 2i + 2.
Padre = floor((i−1)/2), si i > 0.

## Ejemplo y análisis

Arreglo del ejemplo:
[50,30,40,10,20,35].

El nodo 40 ocupa i = 2.
Su hijo izquierdo ocupa 5: valor 35.
Su hijo derecho sería 6, pero no existe porque n = 6.

Siempre verificar que el índice sea menor que el tamaño activo.

# 5. Un heap no es una lista ordenada

## Concepto

La propiedad del heap establece un orden parcial. La raíz contiene un extremo, pero el resto del arreglo no tiene por qué estar ordenado.

Por esa razón, una búsqueda binaria convencional no puede aplicarse directamente al arreglo de un heap.

### Ejemplo y análisis

[50,30,40,10,20,35] es un max-heap válido y no está ordenado.

Los valores 30 y 40 pueden aparecer en ese orden porque son hermanos. La propiedad se compara con el padre, no con el vecino del arreglo.

Buscar una clave arbitraria puede exigir O(n).

# 6. Inserción: subir hasta recuperar la propiedad

## Concepto

La nueva clave se coloca al final del arreglo para conservar la forma completa. Luego se compara con su padre y asciende mientras viole la propiedad.

Este ajuste se conoce como sift-up o ascenso. Recorre como máximo la altura del árbol.

## Ejemplo y análisis

Max-heap inicial:
[50,30,40,10,20,35].
Insertar 45:
[50,30,40,10,20,35,45].

45 supera a su padre 40: se intercambian.
Resultado: [50,30,45,10,20,35,40].
45 no supera a 50, así que termina.

# 7. Extracción de la raíz: descender

## Concepto

Para extraer el extremo se guarda la raíz y se sustituye por el último elemento. Se reduce el tamaño y se realiza un descenso.

En un max-heap se compara con el hijo de mayor valor; en un min-heap, con el de menor valor. Se intercambia mientras sea necesario.

## Ejemplo y análisis

Max-heap inicial:
[50,30,40,10,20,35].
Extraer 50 y mover 35 a la raíz:
[35,30,40,10,20].

El hijo mayor es 40: intercambiar.
Resultado: [40,30,35,10,20].
El extremo extraído fue 50.

# 8. Construcción de un heap desde un arreglo

## Concepto

Es posible insertar n elementos uno a uno, pero la construcción de abajo hacia arriba utiliza una estrategia más eficiente.

Se aplica descenso desde el último nodo interno, floor(n/2)−1, hasta la raíz. Las hojas ya cumplen la propiedad de forma individual.

### Ejemplo y análisis

Entrada: [4,10,3,5,1].
Último interno: i = 1.
Nodo 10: ya domina a 5 y 1.
En i = 0, el 4 desciende.

Max-heap resultante: [10,5,3,4,1].
Construcción bottom-up: O(n), aunque un descenso individual puede costar O(log n).

# 9. Min-heap: acceso al menor elemento

## Concepto

En un min-heap, la raíz contiene una clave mínima. Cada nodo es menor o igual que sus hijos.

La estructura es útil cuando el menor valor representa la mayor prioridad: una fecha próxima o un tiempo de espera reducido.

### Ejemplo y análisis

Ejemplo válido:
[5,10,8,20,15,12].

Raíz mínima: 5.
10 ≤ 20 y 15.
8 ≤ 12.

Consultar el mínimo cuesta O(1).
Extraerlo requiere reorganizar y cuesta O(log n) en el peor caso.

# 10. Max-heap: acceso al mayor elemento

## Concepto

En un max-heap, la raíz contiene una clave máxima. Cada nodo es mayor o igual que sus hijos.

Sirve cuando una puntuación mayor representa mayor prioridad. La elección del tipo depende del significado de la clave y de la comparación.

### Ejemplo y análisis

Ejemplo válido:
[50,30,40,10,20,35].

Raíz máxima: 50.
30 ≥ 10 y 20.
40 ≥ 35.

Consultar el máximo cuesta O(1).
Extraerlo puede exigir O(log n) para restaurar la propiedad.

# 11. Comparación de los dos tipos

| Característica | Min-heap | Max-heap | Interpretación |
|---|---|---|---|
| Regla | Padre ≤ hijos | Padre ≥ hijos | Relación local |
| Raíz | Mínimo | Máximo | Extremo accesible |
| Ascenso | Sube el menor | Sube el mayor | Tras insertar |
| Descenso | Elegir hijo menor | Elegir hijo mayor | Tras extraer |
| Ejemplo de prioridad | Fecha más próxima | Puntaje más alto | Depende del problema |

Ambos conservan la misma forma de árbol completo y utilizan la misma representación por niveles.

# 12. Cómo identificar si un arreglo es un heap

## Concepto

Para verificar un heap binario basta con revisar cada nodo interno y los hijos que existan. La forma completa viene dada por el arreglo sin huecos dentro del tamaño activo.

La verificación completa visita O(n) relaciones. Revisar solamente la raíz no garantiza que todo el heap sea válido.

## Ejemplo y análisis

[9,7,8,3,6] es un max-heap.
9 ≥ 7 y 8; 7 ≥ 3 y 6.

[9,7,8,3,10] no lo es.
Aunque 9 domina a sus hijos, el 7 no domina al 10.

Una violación en cualquier nivel invalida la propiedad.

# 13. Claves repetidas y desempates

## Concepto

La igualdad está permitida: un padre puede tener la misma prioridad que uno de sus hijos.

Un heap convencional no garantiza conservar el orden de llegada de elementos con igual prioridad. Si ese orden es necesario, debe incorporarse explícitamente en la comparación.

## Ejemplo y análisis

Tareas con prioridad 2:
Tarea A, llegada 1.
Tarea B, llegada 2.

En un min-heap puede compararse el par (prioridad, llegada). Primero se atiende la menor prioridad numérica y, si empatan, la llegada más antigua. La regla debe ser consistente.

# 14. Operaciones y costos de un heap binario

| Operación | Costo | Condición | Resultado |
|---|---|---|---|
| Consultar raíz | O(1) | Heap no vacío | Mínimo o máximo |
| Insertar | O(log n) | Capacidad disponible | Ajuste por ascenso |
| Extraer raíz | O(log n) | Heap no vacío | Ajuste por descenso |
| Construir bottom-up | O(n) | Arreglo inicial | Heap válido |
| Buscar clave arbitraria | O(n) peor caso | Sin índice adicional | No hay orden total |

Si el arreglo dinámico necesita crecer, una inserción puede además copiar O(n) elementos; el ajuste del heap sigue siendo O(log n).

# 15. Qué significa ordenar

## Concepto

Ordenar consiste en reorganizar los elementos para que cumplan un criterio: ascendente, descendente o definido por varias claves.

Además de producir un orden correcto, el algoritmo debe conservar los elementos originales con sus cantidades, sin perder ni crear registros.

## Ejemplo y análisis

Entrada: [7,3,5,3].
Orden ascendente: [3,3,5,7].

Los dos valores 3 deben conservarse.

Para registros, puede ordenarse por nota y luego por carné. Es necesario definir el criterio antes de elegir o implementar el algoritmo.

# 16. Cómo comparar algoritmos de ordenamiento

## Concepto

El tiempo de ejecución no es el único criterio. También se analizan memoria auxiliar, estabilidad, tamaño de entrada y organización previa de los datos.

Un algoritmo estable conserva el orden relativo de elementos con claves iguales. La estabilidad se refiere a los registros, no solo a números repetidos.

## Ejemplo y análisis

Registros: (Ana,80), (Luis,70), (Eva,80).

Al ordenar de forma estable por nota: (Luis,70), (Ana,80), (Eva,80).

Ana sigue antes que Eva. Si la aplicación necesita ese orden, debe elegirse una implementación estable o definir un desempate explícito.

# 17. Burbuja: intercambiar vecinos

## Concepto

Burbuja compara elementos adyacentes e intercambia los que están fuera de orden. En cada pasada ascendente, un elemento grande llega al extremo derecho del segmento activo.

Puede detenerse si una pasada completa no produce intercambios. Esa mejora beneficia las entradas ya ordenadas.

## Ejemplo y análisis

Entrada: [5,1,4,2].
Primera pasada:
5 y 1 → [1,5,4,2].
5 y 4 → [1,4,5,2].
5 y 2 → [1,4,2,5].

El 5 queda al final. Se repite sobre el prefijo restante hasta completar el orden.

# 18. Inserción: ampliar un prefijo ordenado

## Concepto

El ordenamiento por inserción mantiene un prefijo ordenado. Toma el siguiente elemento, desplaza los mayores e inserta la clave en su posición.

Conserva el orden de iguales si solo desplaza valores estrictamente mayores. Puede funcionar bien con entradas pequeñas o casi ordenadas.

## Ejemplo y análisis

Entrada: [5,2,4,1].
Insertar 2 en [5] → [2,5,4,1].
Insertar 4 en [2,5] → [2,4,5,1].
Insertar 1 → [1,2,4,5].

Peor caso: $O(n^2)$.
Mejor caso ya ordenado: $O(n)$.

# 19. Selección: elegir el siguiente mínimo

## Concepto

Selección busca el menor elemento del segmento pendiente y lo coloca en su posición definitiva.

Reduce el número de intercambios, pero sigue buscando el mínimo en cada pasada. Su versión convencional utiliza $O(n^2)$ comparaciones, incluso con datos ya ordenados.

## Ejemplo y análisis

Entrada: [4,2,5,1].
Mínimo de todo el arreglo: 1.
Intercambiar con 4 → [1,2,5,4].

En el sufijo [2,5,4], el mínimo es 2.
Después se coloca 4 → [1,2,4,5].

La versión basada en intercambios no es estable en general.

# 20. Merge sort: dividir y fusionar

## Concepto

Merge sort divide la entrada en partes, ordena cada parte y fusiona dos secuencias ordenadas.

En su implementación habitual sobre arreglos utiliza memoria auxiliar O(n) y tiempo O(n log n). La fusión puede ser estable si los empates se toman primero de la parte izquierda.

### Ejemplo y análisis

Entrada: [8,3,6,2].
Dividir: [8,3] y [6,2].
Ordenar: [3,8] y [2,6].
Fusionar:
Tomar 2, después 3, después 6 y finalmente 8.

Resultado: [2,3,6,8]. La fusión recorre las dos partes de forma secuencial.

# 21. Quicksort: particionar alrededor de un pivote

## Concepto

Quicksort selecciona un pivote y reorganiza el segmento para separar valores según su relación con él. Luego resuelve las partes restantes.

Su rendimiento depende de la partición y del equilibrio de los tamaños. Con pivotes aleatorios tiene tiempo esperado O(n log n), pero peor caso $O(n^2)$.

## Ejemplo y análisis

Ejemplo conceptual:
Entrada: [7,2,9,4,1], pivote 4.
Menores: [2,1].
Igual: [4].
Mayores: [7,9].

Al ordenar las partes: [1,2,4,7,9]. Esta separación conceptual no representa necesariamente la memoria usada por una implementación in-place.

# 22. Heapsort: ordenar a partir de un max-heap

## Concepto

Heapsort construye un max-heap y lleva repetidamente su raíz al final del segmento activo. Después reduce el tamaño del heap y aplica descenso.

Con un max-heap se obtiene orden ascendente: cada extracción fija el mayor restante en su posición final.

### Ejemplo y análisis

Entrada: [4,10,3,5,1].
Construir max-heap: [10,5,3,4,1].

Intercambiar 10 con el último valor: el 10 queda fuera del heap activo. Restaurar el prefijo y repetir.

El arreglo se divide en un heap pendiente y un sufijo ya ordenado.

# 23. Traza de heapsort

## Concepto

En cada paso se extrae el máximo hacia el sufijo y se restaura la propiedad del prefijo. El sufijo deja de participar en las comparaciones del heap.

El tamaño activo es esencial: incluir accidentalmente el sufijo en el descenso puede destruir el orden conseguido.

## Ejemplo y análisis

Heap inicial: [10,5,3,4,1].
Después de fijar 10: [5,4,3,1 | 10].
Después de fijar 5: [4,1,3 | 5,10].
Después de fijar 4: [3,1 | 4,5,10].
Después de fijar 3: [1 | 3,4,5,10].

Resultado: [1,3,4,5,10].

# 24. Pseudocódigo de heapsort

## Concepto

La construcción inicial es O(n). Las extracciones y descensos cuestan en conjunto O(n log n) en el peor caso.

La implementación usual con descenso iterativo puede ordenar en el mismo arreglo con O(1) memoria auxiliar. Heapsort no es estable en su versión convencional.

## Pseudocódigo

```
heapsort(a):
  n = longitud(a)
  construir_max_heap(a,n)
  para fin = n-1 hasta 1:
    intercambiar(a[0],a[fin])
    descender(a,0,fin)

El límite fin no se incluye:
solo a[0:fin] sigue siendo heap.
```

# 25. Comparativa de ordenamientos

| Algoritmo | Mejor caso | Peor caso | Estable usual |
|---|---|---|---|
| Burbuja con corte | O(n) | O(n²) | Sí |
| Inserción | O(n) | O(n²) | Sí |
| Selección | O(n²) | O(n²) | No |
| Merge sort | O(n log n) | O(n log n) | Sí |
| Quicksort estándar | O(n log n) | O(n²) | No |
| Heapsort | O(n log n)* | O(n log n) | No |

* Cota general superior; puede haber entradas especiales con menos trabajo. Propiedades de las versiones convencionales; los detalles de implementación importan.

# 26. Elegir un ordenamiento según el problema

## Concepto

Una entrada pequeña o casi ordenada puede favorecer inserción. Una necesidad de estabilidad puede favorecer merge sort. Un límite estricto de memoria auxiliar puede favorecer heapsort iterativo.

La elección requiere comparar propiedades y medir con datos representativos; no basta con nombrar un algoritmo.

## Ejemplo y análisis

Tres preguntas:
1. ¿Importa el orden de los empates?
2. ¿Cuánta memoria auxiliar hay?
3. ¿Qué tamaño y patrón tiene la entrada?

Si se elige quicksort, considerar la política de pivote y el peor caso. Si se elige heapsort, verificar que la estabilidad no sea un requisito.

# 27. Qué significa buscar

## Concepto

Buscar consiste en localizar un elemento que cumpla una condición o confirmar su ausencia. Debe definirse qué devolver: un índice, un registro, la primera coincidencia o todas ellas.

La organización de los datos determina qué algoritmo puede aplicarse de forma correcta y eficiente.

### Ejemplo y análisis

En [4,7,7,9], buscar 7 tiene dos coincidencias.

Una búsqueda puede devolver el índice 1 o 2 si solo se pide cualquier aparición. Si se requiere la primera, debe establecerse esa condición y ajustar el algoritmo.

El valor de retorno para ausencia también debe estar definido.

# 28. Búsqueda secuencial

## Concepto

La búsqueda secuencial revisa las entradas una a una hasta encontrar la clave o agotar la colección. No exige que el arreglo esté ordenado.

Su mejor caso es O(1), si encuentra la clave al inicio. Su peor caso es O(n), para una ausencia o una coincidencia al final.

### Ejemplo y análisis

Entrada: [8,3,10,5].
Buscar 10:
8: no coincide.
3: no coincide.
10: encontrado en índice 2.

Tres comparaciones de igualdad. Para buscar 7, deben revisarse las cuatro entradas antes de confirmar ausencia.

# 29. Búsqueda binaria y su requisito

## Concepto

La búsqueda binaria mantiene un intervalo candidato en un arreglo ordenado. Compara la clave con el elemento central y descarta la mitad que no puede contenerla.

El criterio de orden debe ser el mismo que usa la búsqueda. Aplicarla a un heap o arreglo desordenado puede producir respuestas incorrectas.

## Ejemplo y análisis

Entrada ordenada: [2,5,8,12,16,23,38].
Buscar 23:
Centro inicial: 12. Como 23 > 12, descartar la mitad izquierda.
Nuevo centro: 23. Coincide.

El intervalo se reduce de forma geométrica: peor caso O(log n).

# 30. Traza de búsqueda binaria fallida

## Concepto

Si la clave no existe, el intervalo termina vacío. Debe actualizarse el límite para excluir el centro ya examinado.

Con límites inclusivos, la búsqueda continúa mientras izquierda ≤ derecha. Mezclar esta convención con intervalos semiabiertos causa errores de límites.

## Ejemplo y análisis

Arreglo: [2,5,8,12,16,23,38].
Buscar 15.
[0,6], centro 3: 12 → ir a la derecha.
[4,6], centro 5: 23 → ir a la izquierda.
[4,4], centro 4: 16 → ir a la izquierda.

Queda izquierda = 4 y derecha = 3.
Resultado: ausente.

# 31. Pseudocódigo de búsqueda binaria

## Concepto

El punto medio puede calcularse como izquierda + floor((derecha−izquierda)/2). Esto evita sumar directamente dos índices grandes.

El algoritmo siguiente devuelve cualquier coincidencia. Encontrar la primera o última aparición requiere una variante adicional.

## Pseudocódigo

```
binaria(a,x):
  izq = 0; der = longitud(a)-1
  mientras izq <= der:
    m = izq + floor((der-izq)/2)
    si a[m] == x: retornar m
    si a[m] < x: izq = m+1
    si no: der = m-1
  retornar -1
```

# 32. Comparación de búsquedas

| Aspecto | Secuencial | Binaria | Implicación |
|---|---|---|---|
| Requisito de orden | No | Sí | Comprobar antes |
| Mejor caso | O(1) | O(1) | Coincidencia inmediata |
| Peor caso | O(n) | O(log n) | Costo por consulta |
| Acceso a datos | Secuencial | Aleatorio eficiente | Arreglo indexable |
| Uso adecuado | Pocas consultas | Muchas consultas ordenadas | Contar preparación |

La cota O(log n) supone acceso eficiente al elemento central; una lista enlazada no ofrece el mismo costo que un arreglo.

# 33. Costo de ordenar antes de buscar

## Concepto

Para una sola búsqueda, ordenar primero puede ser más costoso que recorrer la lista. Si habrá muchas consultas sobre datos que cambian poco, ordenar puede compensar.

Hay que contar tanto la preparación como las consultas y el mantenimiento de ese orden cuando llegan datos nuevos.

## Ejemplo y análisis

Con q consultas y n datos:
Secuencial: O(qn).
Ordenar y buscar: O(n log n + q log n).

Estas expresiones comparan órdenes de crecimiento, no tiempos exactos. El beneficio depende de q, de las constantes y de cuánto cambie la colección.

# 34. Aplicación: priorizar tareas

## Concepto

Un heap permite atender la siguiente tarea según una prioridad definida. Una prioridad numérica menor puede representar atención más temprana mediante un min-heap.

La estructura mantiene el extremo disponible sin ordenar toda la colección después de cada inserción.

### Ejemplo y análisis

Tareas: A(3), B(1), C(2).
Primera en atenderse: B.
Después: C y luego A.

Si llega D(0) antes de la siguiente extracción, debe ascender y quedar disponible en la raíz. La comparación decide la prioridad, no la posición original en la lista.

# 35. Aplicación: ordenar registros

## Concepto

Ordenar registros exige mover cada registro completo, no únicamente su clave. También deben respetarse las reglas de comparación y los empates.

En una lista de estudiantes, ordenar las notas separadas de los nombres rompería las asociaciones y produciría datos incorrectos.

### Ejemplo y análisis

Entrada: Ana(85), Luis(70), Eva(85).

Orden ascendente estable por nota: Luis(70), Ana(85), Eva(85).

Para conservar Ana antes que Eva, puede usarse merge sort estable. Si se decide desempatar por nombre, ese segundo criterio debe definirse explícitamente.

# 36. Aplicación: consultar un catálogo

## Concepto

Un catálogo estático ordenado por código permite consultas con búsqueda binaria. Si se consulta por otra propiedad, como precio, ese orden por código no garantiza una búsqueda correcta sobre precios.

Cada búsqueda debe usar una organización compatible con su criterio.

## Ejemplo y análisis

Códigos: [10,20,35,50,80].
Buscar 50: centro 35, después 50.

Si el catálogo recibe nuevos registros, debe mantenerse el orden o reconstruirlo antes de usar la búsqueda binaria. Para una consulta aislada en una lista desordenada puede bastar un recorrido.

# 37. Ejercicio: reconocer y modificar un heap

## Concepto

Determinar el tipo de heap de [20,15,18,8,10,12]. Después insertar 25 y extraer la raíz del resultado.

Verificar las relaciones padre-hijo después de cada paso, sin exigir que el arreglo quede ordenado.

## Ejemplo y análisis

Es un max-heap.
Insertar 25 y ascender:
[25,15,20,8,10,12,18].

Extraer 25: llevar 18 a la raíz y descender.
Resultado: [20,15,18,8,10,12].

Ambos pasos mantienen la forma completa y la propiedad de máximo.

# 38. Ejercicio: ordenar y luego buscar

## Concepto

Ordenar [9,4,7,1,3] en forma ascendente mediante inserción. Después buscar 7 con búsqueda binaria.

Registrar los cambios y comparar la solución con un compañero. Escuchar diferencias de procedimiento sin descalificar y comprobar el resultado con la misma entrada.

## Ejemplo y análisis

Pasos de inserción:
[4,9,7,1,3]
[4,7,9,1,3]
[1,4,7,9,3]
[1,3,4,7,9]

Buscar 7: centro índice 2 → 4.
Ir al intervalo [3,4].
Centro índice 3 → 7. Encontrado.

# 39. Justificar la solución elegida

## Concepto

Una explicación completa identifica el problema, verifica las precondiciones y argumenta la elección del algoritmo. Después comprueba que la salida sea correcta.

Comparar soluciones permite aprender de enfoques diferentes. La tolerancia exige escuchar; la verificación permite determinar si un procedimiento funciona.

## Ejemplo y análisis

Lista de revisión:
• ¿Heap mínimo o máximo según prioridad?
• ¿El ordenamiento conserva registros y duplicados?
• ¿La búsqueda binaria recibe datos ordenados?
• ¿Se consideran tiempo y memoria?
• ¿Se han probado ausencia, repetidos y entrada vacía?

# Conceptos clave aprendidos

Definición de Heaps
Tipos de Heaps
Propiedad de mínimo o máximo
Representación por niveles

Algoritmos de ordenamiento
Algoritmos de búsqueda
Aplicación de algoritmos
Tolerancia al comparar soluciones

# Referencias

## Heaps y ordenamiento

Robert Sedgewick y Kevin Wayne. Algorithms, 4th edition.

Priority Queues.
https://algs4.cs.princeton.edu/24pq/

Sorting.
https://algs4.cs.princeton.edu/20sorting/

## Búsqueda y aplicaciones

Princeton University.
Introduction to Programming.

Sorting and Searching.
https://introcs.cs.princeton.edu/python/42sort/

# Recursos y práctica

## Revisión del material

Revisar los ejemplos de heaps y ordenamiento del material de referencia.

Repetir las trazas con entradas pequeñas y comprobar cada relación padre-hijo, cada intercambio y cada límite de búsqueda.

### Alcance de la semana

Usar únicamente los cinco subtemas de la planificación:

1. Definición de Heaps.
2. Tipos de Heaps.
3. Algoritmos de ordenamiento.
4. Algoritmos de búsqueda.
5. Aplicación de algoritmos.

# PAUSA DE HIDRATACIÓN

## DUDAS

¿En qué se diferencian min-heap y max-heap?
¿Por qué un heap no admite búsqueda binaria directa?
¿Cómo aplicamos la tolerancia al discutir soluciones?